PEP 695: Type Parameter Syntax

Status: Final · Created: 15-Jun-2022 · Public domain / CC0

Abstract

This PEP specifies an improved syntax for specifying type parameters within a generic class, function, or type alias. It also introduces a new statement for declaring type aliases.

Motivation

484 introduced type variables into the language. 612 built upon this concept by introducing parameter specifications, and 646 added variadic type variables.

While generic types and type parameters have grown in popularity, the syntax for specifying type parameters still feels "bolted on" to Python. This is a source of confusion among Python developers.

There is consensus within the Python static typing community that it is time to provide a formal syntax that is similar to other modern programming languages that support generic types.

An analysis of 25 popular typed Python libraries revealed that type variables (in particular, the typing.TypeVar symbol) were used in 14% of modules.

Points of Confusion

While the use of type variables has become widespread, the manner in which they are specified within code is the source of confusion among many Python developers. There are a couple of factors that contribute to this confusion.

The scoping rules for type variables are difficult to understand. Type variables are typically allocated within the global scope, but their semantic meaning is valid only when used within the context of a generic class, function, or type alias. A single runtime instance of a type variable may be reused in multiple generic contexts, and it has a different semantic meaning in each of these contexts. This PEP proposes to eliminate this source of confusion by declaring type parameters at a natural place within a class, function, or type alias declaration statement.

Generic type aliases are often misused because it is not clear to developers that a type argument must be supplied when the type alias is used. This leads to an implied type argument of Any, which is rarely the intent. This PEP proposes to add new syntax that makes generic type alias declarations clear.

483 and 484 introduced the concept of "variance" for a type variable used within a generic class. Type variables can be invariant, covariant, or contravariant. The concept of variance is an advanced detail of type theory that is not well understood by most Python developers, yet they must confront this concept today when defining their first generic class. This PEP largely eliminates the need for most developers to understand the concept of variance when defining generic classes.

When more than one type parameter is used with a generic class or type alias, the rules for type parameter ordering can be confusing. It is normally based on the order in which they first appear within a class or type alias declaration statement. However, this can be overridden in a class definition by including a "Generic" or "Protocol" base class. For example, in the class declaration class ClassA(Mapping[K, V]), the type parameters are ordered as K and then V. However, in the class declaration class ClassB(Mapping[K, V], Generic[V, K]), the type parameters are ordered as V and then K. This PEP proposes to make type parameter ordering explicit in all cases.

The practice of sharing a type variable across multiple generic contexts creates other problems today. Modern editors provide features like "find all references" and "rename all references" that operate on symbols at the semantic level. When a type parameter is shared among multiple generic classes, functions, and type aliases, all references are semantically equivalent.

Type variables defined within the global scope also need to be given a name that starts with an underscore to indicate that the variable is private to the module. Globally-defined type variables are also often given names to indicate their variance, leading to cumbersome names like "_T_contra" and "_KT_co". The current mechanisms for allocating type variables also requires the developer to supply a redundant name in quotes (e.g. T = TypeVar("T")). This PEP eliminates the need for the redundant name and cumbersome variable names.

Defining type parameters today requires importing the TypeVar and Generic symbols from the typing module. Over the past several releases of Python, efforts have been made to eliminate the need to import typing symbols for common use cases, and the PEP furthers this goal.

Summary Examples

Defining a generic class prior to this PEP looks something like this.

::

```
from typing import Generic, TypeVar
_T_co = TypeVar("_T_co", covariant=True, bound=str)
class ClassA(Generic[_T_co]):
    def method1(self) -> _T_co:
        ...
```

With the new syntax, it looks like this.

::

```
class ClassA[T: str]:
    def method1(self) -> T:
        ...
```

Here is an example of a generic function today.

::

```
from typing import TypeVar
_T = TypeVar("_T")
def func(a: _T, b: _T) -> _T:
    ...
```

And the new syntax.

::

```
def func[T](a: T, b: T) -> T:
    ...
```

Here is an example of a generic type alias today.

::

```
from typing import TypeAlias
_T = TypeVar("_T")
ListOrSet: TypeAlias = list[_T] | set[_T]
```

And with the new syntax.

::

```
type ListOrSet[T] = list[T] | set[T]
```

Specification

Type Parameter Declarations

Here is a new syntax for declaring type parameters for generic classes, functions, and type aliases. The syntax adds support for a comma-delimited list of type parameters in square brackets after the name of the class, function, or type alias.

Simple (non-variadic) type variables are declared with an unadorned name. Variadic type variables are preceded by * (see 646 for details). Parameter specifications are preceded by ** (see 612 for details).

::

```
# This generic class is parameterized by a TypeVar T, a
# TypeVarTuple Ts, and a ParamSpec P.
class ChildClass[T, *Ts, **P]: ...
```

There is no need to include Generic as a base class. Its inclusion as a base class is implied by the presence of type parameters, and it will automatically be included in the __mro__ and __orig_bases__ attributes for the class. The explicit use of a Generic base class will result in a runtime error.

::

```
class ClassA[T](Generic[T]): ...  # Runtime error
```

A Protocol base class with type arguments may generate a runtime error. Type checkers should generate an error in this case because the use of type arguments is not needed, and the order of type parameters for the class are no longer dictated by their order in the Protocol base class.

::

```
class ClassA[S, T](Protocol): ... # OK
class ClassB[S, T](Protocol[S, T]): ... # Recommended type checker error
```

Type parameter names within a generic class, function, or type alias must be unique within that same class, function, or type alias. A duplicate name generates a syntax error at compile time. This is consistent with the requirement that parameter names within a function signature must be unique.

::

```
class ClassA[T, *T]: ... # Syntax Error
def func1[T, **T](): ... # Syntax Error
```

Class type parameter names are mangled if they begin with a double underscore, to avoid complicating the name lookup mechanism for names used within the class. However, the __name__ attribute of the type parameter will hold the non-mangled name.

Upper Bound Specification

For a non-variadic type parameter, an "upper bound" type can be specified through the use of a type annotation expression. If an upper bound is not specified, the upper bound is assumed to be object.

::

```
class ClassA[T: str]: ...
```

The specified upper bound type must use an expression form that is allowed in type annotations. More complex expression forms should be flagged as an error by a type checker. Quoted forward references are allowed.

The specified upper bound type must be concrete. An attempt to use a generic type should be flagged as an error by a type checker. This is consistent with the existing rules enforced by type checkers for a TypeVar constructor call.

::

```
class ClassA[T: dict[str, int]]: ...  # OK
class ClassB[T: "ForwardReference"]: ...  # OK
class ClassC[V]:
    class ClassD[T: dict[str, V]]: ...  # Type checker error: generic type
class ClassE[T: [str, int]]: ...  # Type checker error: illegal expression form
```

Constrained Type Specification

484 introduced the concept of a "constrained type variable" which is constrained to a set of two or more types. The new syntax supports this type of constraint through the use of a literal tuple expression that contains two or more types.

::

```
class ClassA[AnyStr: (str, bytes)]: ...  # OK
class ClassB[T: ("ForwardReference", bytes)]: ...  # OK
class ClassC[T: ()]: ...  # Type checker error: two or more types required
class ClassD[T: (str, )]: ...  # Type checker error: two or more types required
t1 = (bytes, str)
class ClassE[T: t1]: ...  # Type checker error: literal tuple expression required
```

If the specified type is not a tuple expression or the tuple expression includes complex expression forms that are not allowed in a type annotation, a type checker should generate an error. Quoted forward references are allowed.

::

```
class ClassF[T: (3, bytes)]: ...  # Type checker error: invalid expression form
```

The specified constrained types must be concrete. An attempt to use a generic type should be flagged as an error by a type checker. This is consistent with the existing rules enforced by type checkers for a TypeVar constructor call.

::

```
class ClassG[T: (list[S], str)]: ...  # Type checker error: generic type
```

Runtime Representation of Bounds and Constraints

The upper bounds and constraints of TypeVar objects are accessible at runtime through the __bound__ and __constraints__ attributes. For TypeVar objects defined through the new syntax, these attributes become lazily evaluated, as discussed under Lazy Evaluation_ below.

Generic Type Alias

We propose to introduce a new statement for declaring type aliases. Similar to class and def statements, a type statement defines a scope for type parameters.

::

```
# A non-generic type alias
type IntOrStr = int | str
# A generic type alias
type ListOrSet[T] = list[T] | set[T]
```

Type aliases can refer to themselves without the use of quotes.

::

```
# A type alias that includes a forward reference
type AnimalOrVegetable = Animal | "Vegetable"
# A generic self-referential type alias
type RecursiveList[T] = T | list[RecursiveList[T]]
```

The type keyword is a new soft keyword. It is interpreted as a keyword only in this part of the grammar. In all other locations, it is assumed to be an identifier name.

Type parameters declared as part of a generic type alias are valid only when evaluating the right-hand side of the type alias.

As with typing.TypeAlias, type checkers should restrict the right-hand expression to expression forms that are allowed within type annotations. The use of more complex expression forms (call expressions, ternary operators, arithmetic operators, comparison operators, etc.) should be flagged as an error.

Type alias expressions are not allowed to use traditional type variables (i.e. those allocated with an explicit TypeVar constructor call). Type checkers should generate an error in this case.

::

```
T = TypeVar("T")
type MyList = list[T]  # Type checker error: traditional type variable usage
```

We propose to deprecate the existing typing.TypeAlias introduced in 613. The new syntax eliminates its need entirely.

Runtime Type Alias Class

At runtime, a type statement will generate an instance of typing.TypeAliasType. This class represents the type. Its attributes include:

* __name__ is a str representing the name of the type alias * __type_params__ is a tuple of TypeVar, TypeVarTuple, or ParamSpec objects that parameterize the type alias if it is generic * __value__ is the evaluated value of the type alias

All of these attributes are read-only.

The value of the type alias is evaluated lazily (see Lazy Evaluation_ below).

Type Parameter Scopes

When the new syntax is used, a new lexical scope is introduced, and this scope includes the type parameters. Type parameters can be accessed by name within inner scopes. As with other symbols in Python, an inner scope can define its own symbol that overrides an outer-scope symbol of the same name. This section provides a verbal description of the new scoping rules. The Scoping Behavior_ section below specifies the behavior in terms of a translation to near-equivalent existing Python code.

Type parameters are visible to other type parameters declared elsewhere in the list. This allows type parameters to use other type parameters within their definition. While there is currently no use for this capability, it preserves the ability in the future to support upper bound expressions or type argument defaults that depend on earlier type parameters.

A compiler error or runtime exception is generated if the definition of an earlier type parameter references a later type parameter even if the name is defined in an outer scope.

::

```
# The following generates no compiler error, but a type checker
# should generate an error because an upper bound type must be concrete,
# and Sequence[S] is generic. Future extensions to the type system may
```

```
# eliminate this limitation.
class ClassA[S, T: Sequence[S]]: ...
# The following generates no compiler error, because the bound for S
# is lazily evaluated. However, type checkers should generate an error.
class ClassB[S: Sequence[T], T]: ...
```

A type parameter declared as part of a generic class is valid within the class body and inner scopes contained therein. Type parameters are also accessible when evaluating the argument list (base classes and any keyword arguments) that comprise the class definition. This allows base classes to be parameterized by these type parameters. Type parameters are not accessible outside of the class body, including class decorators.

::

```
class ClassA[T](BaseClass[T], param = Foo[T]): ...  # OK
print(T)  # Runtime error: 'T' is not defined
@dec(Foo[T])  # Runtime error: 'T' is not defined
class ClassA[T]: ...
```

A type parameter declared as part of a generic function is valid within the function body and any scopes contained therein. It is also valid within parameter and return type annotations. Default argument values for function parameters are evaluated outside of this scope, so type parameters are not accessible in default value expressions. Likewise, type parameters are not in scope for function decorators.

::

```
def func1[T](a: T) -> T: ...  # OK
print(T)  # Runtime error: 'T' is not defined
def func2[T](a = list[T]): ...  # Runtime error: 'T' is not defined
@dec(list[T])  # Runtime error: 'T' is not defined
def func3[T](): ...
```

A type parameter declared as part of a generic type alias is valid within the type alias expression.

::

```
type Alias1[K, V] = Mapping[K, V] | Sequence[K]
```

Type parameter symbols defined in outer scopes cannot be bound with nonlocal statements in inner scopes.

::

```
S = 0
def outer1[S]():
    S = 1
    T = 1
    def outer2[T]():
        def inner1():
            nonlocal S  # OK because it binds variable S from outer1
            nonlocal T  # Syntax error: nonlocal binding not allowed for type parameter
        def inner2():
            global S  # OK because it binds variable S from global scope
```

The lexical scope introduced by the new type parameter syntax is unlike traditional scopes introduced by a def or class statement. A type parameter scope acts more like a temporary "overlay" to the containing scope. The only new symbols contained within its symbol table are the type parameters defined using the new syntax. References to all other symbols are treated as though they were found within the containing scope. This allows base class lists (in class definitions) and type annotation expressions (in function definitions) to reference symbols defined in the containing scope.

::

```
class Outer:
```

```
    class Private:
        pass
    # If the type parameter scope was like a traditional scope,
    # the base class 'Private' would not be accessible here.
    class Inner[T](Private, Sequence[T]):
        pass
    # Likewise, 'Inner' would not be available in these type annotations.
    def method1[T](self, a: Inner[T]) -> Inner[T]:
        return a
```

The compiler allows inner scopes to define a local symbol that overrides an outer-scoped type parameter.

Consistent with the scoping rules defined in 484, type checkers should generate an error if inner-scoped generic classes, functions, or type aliases reuse the same type parameter name as an outer scope.

::

```
T = 0
@decorator(T)  # Argument expression T evaluates to 0
class ClassA[T](Sequence[T]):
    T = 1
    # All methods below should result in a type checker error
    # "type parameter 'T' already in use" because they are using the
    # type parameter 'T', which is already in use by the outer scope
    # 'ClassA'.
    def method1[T](self):
        ...
    def method2[T](self, x = T):  # Parameter 'x' gets default value of 1
        ...
    def method3[T](self, x: T):  # Parameter 'x' has type T (scoped to method3)
        ...
```

Symbols referenced in inner scopes are resolved using existing rules except that type parameter scopes are also considered during name resolution.

::

```
T = 0
# T refers to the global variable
print(T)  # Prints 0
class Outer[T]:
    T = 1
    # T refers to the local variable scoped to class 'Outer'
    print(T)  # Prints 1
    class Inner1:
        T = 2
        # T refers to the local type variable within 'Inner1'
        print(T)  # Prints 2
        def inner_method(self):
            # T refers to the type parameter scoped to class 'Outer';
            # If 'Outer' did not use the new type parameter syntax,
            # this would instead refer to the global variable 'T'
            print(T)  # Prints 'T'
    def outer_method(self):
        T = 3
        # T refers to the local variable within 'outer_method'
        print(T)  # Prints 3
        def inner_func():
            # T refers to the variable captured from 'outer_method'
            print(T)  # Prints 3
```

When the new type parameter syntax is used for a generic class, assignment expressions are not allowed within the argument list for the class definition. Likewise, with functions that use the new type parameter syntax, assignment expressions are not allowed within parameter or return type annotations, nor are they allowed within the expression that defines a type alias, or within the bounds and constraints of a TypeVar. Similarly, yield, yield from, and await expressions are disallowed in these contexts.

This restriction is necessary because expressions evaluated within the new lexical scope should not introduce symbols within that scope other than the defined type parameters, and should not affect whether the enclosing function is a generator or coroutine.

::

```
class ClassA[T]((x := Sequence[T])): ...  # Syntax error: assignment expression not allowed
def func1[T](val: (x := int)): ...  # Syntax error: assignment expression not allowed
def func2[T]() -> (x := Sequence[T]): ...  # Syntax error: assignment expression not allowed
type Alias1[T] = (x := list[T])  # Syntax error: assignment expression not allowed
```

Accessing Type Parameters at Runtime

A new attribute called __type_params__ is available on generic classes, functions, and type aliases. This attribute is a tuple of the type parameters that parameterize the class, function, or alias. The tuple contains TypeVar, ParamSpec, and TypeVarTuple instances.

Type parameters declared using the new syntax will not appear within the dictionary returned by globals() or locals().

Variance Inference

This PEP eliminates the need for variance to be specified for type parameters. Instead, type checkers will infer the variance of type parameters based on their usage within a class. Type parameters are inferred to be invariant, covariant, or contravariant depending on how they are used.

Python type checkers already include the ability to determine the variance of type parameters for the purpose of validating variance within a generic protocol class. This capability can be used for all classes (whether or not they are protocols) to calculate the variance of each type parameter.

The algorithm for computing the variance of a type parameter is as follows.

For each type parameter in a generic class:

1. If the type parameter is variadic (TypeVarTuple) or a parameter specification (ParamSpec), it is always considered invariant. No further inference is needed.

2. If the type parameter comes from a traditional TypeVar declaration and is not specified as infer_variance (see below), its variance is specified by the TypeVar constructor call. No further inference is needed.

3. Create two specialized versions of the class. We'll refer to these as upper and lower specializations. In both of these specializations, replace all type parameters other than the one being inferred by a dummy type instance (a concrete anonymous class that is type compatible with itself and assumed to meet the bounds or constraints of the type parameter). In the upper specialized class, specialize the target type parameter with an object instance. This specialization ignores the type parameter's upper bound or constraints. In the lower specialized class, specialize the target type parameter with itself (i.e. the corresponding type argument is the type parameter itself).

4. Determine whether lower can be assigned to upper using normal type compatibility rules. If so, the target type parameter is covariant. If not, determine whether upper can be assigned to lower. If so, the target type parameter is contravariant. If neither of these combinations are assignable, the target type parameter is invariant.

Here is an example.

::

```
class ClassA[T1, T2, T3](list[T1]):
    def method1(self, a: T2) -> None:
```

```
        ...
    def method2(self) -> T3:
        ...
```

To determine the variance of T1, we specialize ClassA as follows:

::

```
upper = ClassA[object, Dummy, Dummy]
lower = ClassA[T1, Dummy, Dummy]
```

We find that upper is not assignable to lower using normal type compatibility rules defined in 484. Likewise, lower is not assignable to upper, so we conclude that T1 is invariant.

To determine the variance of T2, we specialize ClassA as follows:

::

```
upper = ClassA[Dummy, object, Dummy]
lower = ClassA[Dummy, T2, Dummy]
```

Since upper is assignable to lower, T2 is contravariant.

To determine the variance of T3, we specialize ClassA as follows:

::

```
upper = ClassA[Dummy, Dummy, object]
lower = ClassA[Dummy, Dummy, T3]
```

Since lower is assignable to upper, T3 is covariant.

Auto Variance For TypeVar

The existing TypeVar class constructor accepts keyword parameters named covariant and contravariant. If both of these are False, the type variable is assumed to be invariant. We propose to add another keyword parameter named infer_variance indicating that a type checker should use inference to determine whether the type variable is invariant, covariant or contravariant. A corresponding instance variable __infer_variance__ can be accessed at runtime to determine whether the variance is inferred. Type variables that are implicitly allocated using the new syntax will always have __infer_variance__ set to True.

A generic class that uses the traditional syntax may include combinations of type variables with explicit and inferred variance.

::

```
T1 = TypeVar("T1", infer_variance=True)  # Inferred variance
T2 = TypeVar("T2")  # Invariant
T3 = TypeVar("T3", covariant=True)  # Covariant
# A type checker should infer the variance for T1 but use the
# specified variance for T2 and T3.
class ClassA(Generic[T1, T2, T3]): ...
```

Compatibility with Traditional TypeVars

The existing mechanism for allocating TypeVar, TypeVarTuple, and ParamSpec is retained for backward compatibility. However, these "traditional" type variables should not be combined with type parameters allocated using the new syntax. Such a combination should be flagged as an error by type checkers. This is necessary because the type parameter order is ambiguous.

It is OK to combine traditional type variables with new-style type parameters if the class, function, or type alias does not use the new syntax. The new-style type parameters must come from an outer scope in this case.

::

```
K = TypeVar("K")
class ClassA[V](dict[K, V]): ...  # Type checker error
class ClassB[K, V](dict[K, V]): ...  # OK
class ClassC[V]:
    # The use of K and V for "method1" is OK because it uses the
    # "traditional" generic function mechanism where type parameters
    # are implicit. In this case V comes from an outer scope (ClassC)
    # and K is introduced implicitly as a type parameter for "method1".
    def method1(self, a: V, b: K) -> V | K: ...
    # The use of M and K are not allowed for "method2". A type checker
    # should generate an error in this case because this method uses the
    # new syntax for type parameters, and all type parameters associated
    # with the method must be explicitly declared. In this case, K
    # is not declared by "method2", nor is it supplied by a new-style
    # type parameter defined in an outer scope.
    def method2[M](self, a: M, b: K) -> M | K: ...
```

Runtime Implementation

Grammar Changes

This PEP introduces a new soft keyword type. It modifies the grammar in the following ways:

1. Addition of optional type parameter clause in class and def statements.

::

```
type_params: '[' t=type_param_seq  ']'
type_param_seq: a[asdl_typeparam_seq*]=','.type_param+ [',']
type_param:
    | a=NAME b=[type_param_bound]
    | '*' a=NAME
    | '**' a=NAME
type_param_bound: ":" e=expression
# Grammar definitions for class_def_raw and function_def_raw are modified
# to reference type_params as an optional syntax element. The definitions
# of class_def_raw and function_def_raw are simplified here for brevity.
class_def_raw: 'class' n=NAME t=[type_params] ...
function_def_raw: a=[ASYNC] 'def' n=NAME t=[type_params] ...
```

2. Addition of new type statement for defining type aliases.

::

```
type_alias: "type" n=NAME t=[type_params] '=' b=expression
```

AST Changes

This PEP introduces a new AST node type called TypeAlias.

::

```
TypeAlias(expr name, typeparam* typeparams, expr value)
```

It also adds an AST node type that represents a type parameter.

::

```
typeparam = TypeVar(identifier name, expr? bound)
    | ParamSpec(identifier name)
    | TypeVarTuple(identifier name)
```

Bounds and constraints are represented identically in the AST. In the implementation, any expression that is a Tuple AST node is treated as a constraint, and any other expression is treated as a bound.

It also modifies existing AST node types FunctionDef, AsyncFunctionDef and ClassDef to include an additional optional attribute called typeparams that includes a list of type parameters associated with the function or class.

Lazy Evaluation

This PEP introduces three new contexts where expressions may occur that represent static types: TypeVar bounds, TypeVar constraints, and the value of type aliases. These expressions may contain references to names that are not yet defined. For example, type aliases may be recursive, or even mutually recursive, and type variable bounds may refer back to the current class. If these expressions were evaluated eagerly, users would need to enclose such expressions in quotes to prevent runtime errors. 563 and 649 detail the problems with this situation for type annotations.

To prevent a similar situation with the new syntax proposed in this PEP, we propose to use lazy evaluation for these expressions, similar to the approach in 649. Specifically, each expression will be saved in a code object, and the code object is evaluated only when the corresponding attribute is accessed (TypeVar.__bound__, TypeVar.__constraints__, or TypeAlias.__value__). After the value is successfully evaluated, the value is saved and later calls will return the same value without re-evaluating the code object.

If 649 is implemented, additional evaluation mechanisms should be added to mirror the options that PEP provides for annotations. In the current version of the PEP, that might include adding an __evaluate_bound__ method to TypeVar taking a format parameter with the same meaning as in PEP 649's __annotate__ method (and a similar __evaluate_constraints__ method, as well as an __evaluate_value__ method on TypeAliasType). However, until PEP 649 is accepted and implemented, only the default evaluation format (PEP 649's "VALUE" format) will be supported.

As a consequence of lazy evaluation, the value observed for an attribute may depend on the time the attribute is accessed.

::

X = int

```
class Foo[T: X, U: X]:
    t, u = T, U
print(Foo.t.__bound__)  # prints "int"
X = str
print(Foo.u.__bound__)  # prints "str"
```

Similar examples affecting type annotations can be constructed using the semantics of PEP 563 or PEP 649.

A naive implementation of lazy evaluation would handle class namespaces incorrectly, because functions within a class do not normally have access to the enclosing class namespace. The implementation will retain a reference to the class namespace so that class-scoped names are resolved correctly.

Scoping Behavior

The new syntax requires a new kind of scope that behaves differently from existing scopes in Python. Thus, the new syntax cannot be described exactly in terms of existing Python scoping behavior. This section specifies these scopes further by reference to existing scoping behavior: the new scopes behave like function scopes, except for a number of minor differences listed below.

All examples include functions introduced with the pseudo-keyword def695. This keyword will not exist in the actual language; it is used to clarify that the new scopes are for the most part like function scopes.

def695 scopes differ from regular function scopes in the following ways:

- If a def695 scope is immediately within a class scope, or within another def695 scope that is immediately within a class scope, then names defined in that class scope can be accessed within the def695 scope. (Regular functions, by contrast, cannot access names defined within an enclosing class scope.) - The following constructs are disallowed directly within a def695 scope, though they may be used within other scopes nested inside a def695 scope:

```
- yield
- yield from
- await
- := (walrus operator)
```

- The qualified name (__qualname__) of objects (classes and functions) defined within def695 scopes is as if the objects were defined within the closest enclosing scope. - Names bound within def695 scopes cannot be rebound with a nonlocal statement in nested scopes.

def695 scopes are used for the evaluation of several new syntactic constructs proposed in this PEP. Some are evaluated eagerly (when a type alias, function, or class is defined); others are evaluated lazily (only when evaluation is specifically requested). In all cases, the scoping semantics are identical:

- Eagerly evaluated values:

- The type parameters of generic type aliases - The type parameters and annotations of generic functions - The type parameters and base class expressions of generic classes - Lazily evaluated values:

```
- The value of generic type aliases
- The bounds of type variables
- The constraints of type variables
```

In the below translations, names that start with two underscores are internal to the implementation and not visible to actual Python code. We use the following intrinsic functions, which in the real implementation are defined directly in the interpreter:

- __make_typealias(*, name, type_params=(), evaluate_value): Creates a new typing.TypeAlias object with the given name, type parameters, and lazily evaluated value. The value is not evaluated until the __value__ attribute is accessed. - __make_typevar_with_bound(*, name, evaluate_bound): Creates a new typing.TypeVar object with the given name and lazily evaluated bound. The bound is not evaluated until the __bound__ attribute is accessed. - __make_typevar_with_constraints(*, name, evaluate_constraints): Creates a new typing.TypeVar object with the given name and lazily evaluated constraints. The constraints are not evaluated until the __constraints__ attribute is accessed.

Non-generic type aliases are translated as follows::

type Alias = int

Equivalent to::

```
def695 __evaluate_Alias():
    return int
```

Alias = __make_typealias(name='Alias', evaluate_value=__evaluate_Alias)

Generic type aliases::

type Alias[T: int] = list[T]

Equivalent to::

```
def695 __generic_parameters_of_Alias():
    def695 __evaluate_T_bound():
```

```
        return int
    T = __make_typevar_with_bound(name='T', evaluate_bound=__evaluate_T_bound)
    def695 __evaluate_Alias():
        return list[T]
    return __make_typealias(name='Alias', type_params=(T,), evaluate_value=__evaluate_Alias)
```

Alias = __generic_parameters_of_Alias()

Generic functions::

```
def f[T](x: T) -> T:
    return x
```

Equivalent to::

```
def695 __generic_parameters_of_f():
    T = typing.TypeVar(name='T')
    def f(x: T) -> T:
        return x
    f.__type_params__ = (T,)
    return f
```

f = __generic_parameters_of_f()

A fuller example of generic functions, illustrating the scoping behavior of defaults, decorators, and bounds. Note that this example does not use ParamSpec correctly, so it should be rejected by a static type checker. It is however valid at runtime, and it us used here to illustrate the runtime semantics.

::

```
@decorator
def f[T: int, U: (int, str), *Ts, **P](
    x: T = SOME_CONSTANT,
    y: U,
    *args: *Ts,
    **kwargs: P.kwargs,
) -> T:
    return x
```

Equivalent to::

```
__default_of_x = SOME_CONSTANT  # evaluated outside the def695 scope
def695 __generic_parameters_of_f():
    def695 __evaluate_T_bound():
        return int
    T = __make_typevar_with_bound(name='T', evaluate_bound=__evaluate_T_bound)
    def695 __evaluate_U_constraints():
        return (int, str)
    U = __make_typevar_with_constraints(name='U', evaluate_constraints=__evaluate_U_constraints)
    Ts = typing.TypeVarTuple("Ts")
    P = typing.ParamSpec("P")
    def f(x: T = __default_of_x, y: U, *args: *Ts, **kwargs: P.kwargs) -> T:
        return x
    f.__type_params__ = (T, U, Ts, P)
    return f
```

f = decorator(__generic_parameters_of_f())

Generic classes::

```
class C[T](Base):
    def __init__(self, x: T):
        self.x = x
```

Equivalent to::

```
def695 __generic_parameters_of_C():
    T = typing.TypeVar('T')
    class C(Base):
        __type_params__ = (T,)
        def __init__(self, x: T):
            self.x = x
   return C
```

C = __generic_parameters_of_C()

The biggest divergence from existing behavior for def695 scopes is the behavior within class scopes. This divergence is necessary so that generics defined within classes behave in an intuitive way::

```
class C:
    class Nested: ...
    def generic_method[T](self, x: T, y: Nested) -> T: ...
```

Equivalent to::

```
class C:
    class Nested: ...
    def695 __generic_parameters_of_generic_method():
        T = typing.TypeVar('T')
        def generic_method(self, x: T, y: Nested) -> T: ...
        return generic_method
    generic_method = __generic_parameters_of_generic_method()
```

In this example, the annotations for x and y are evaluated within a def695 scope, because they need access to the type parameter T for the generic method. However, they also need access to the Nested name defined within the class namespace. If def695 scopes behaved like regular function scopes, Nested would not be visible within the function scope. Therefore, def695 scopes that are immediately within class scopes have access to that class scope, as described above.

Library Changes

Several classes in the typing module that are currently implemented in Python must be partially implemented in C. This includes TypeVar, TypeVarTuple, ParamSpec, and Generic, and the new class TypeAliasType (described above). The implementation may delegate to the Python version of typing.py for some behaviors that interact heavily with the rest of the module. The documented behaviors of these classes should not change.

Reference Implementation

This proposal is prototyped in CPython PR #103764.

The Pyright type checker supports the behavior described in this PEP.

Rejected Ideas

Prefix Clause

We explored various syntactic options for specifying type parameters that preceded def and class statements. One such variant we considered used a using clause as follows:

::

```
using S, T
class ClassA: ...
```

This option was rejected because the scoping rules for the type parameters were less clear. Also, this syntax did not interact well with class and function decorators, which are common in Python.

Only one other popular programming language, C++, uses this approach.

We likewise considered prefix forms that looked like decorators (e.g., @using(S, T)). This idea was rejected because such forms would be confused with regular decorators, and they would not compose well with existing decorators. Furthermore, decorators are logically executed after the statement they are decorating, so it would be confusing for them to introduce symbols (type parameters) that are visible within the "decorated" statement, which is logically executed before the decorator itself.

Angle Brackets

Many languages that support generics make use of angle brackets. (Refer to the table at the end of Appendix A for a summary.) We explored the use of angle brackets for type parameter declarations in Python, but we ultimately rejected it for two reasons. First, angle brackets are not considered "paired" by the Python scanner, so end-of-line characters between a < and > token are retained. That means any line breaks within a list of type parameters would require the use of unsightly and cumbersome \ escape sequences. Second, Python has already established the use of square brackets for explicit specialization of a generic type (e.g., list[int]). We concluded that it would be inconsistent and confusing to use angle brackets for generic declarations but square brackets for explicit specialization. All other languages that we surveyed were consistent in this regard.

Bounds Syntax

We explored various syntactic options for specifying the bounds and constraints for a type variable. We considered, but ultimately rejected, the use of a <: token like in Scala, the use of an extends or with keyword like in various other languages, and the use of a function call syntax similar to today's typing.TypeVar constructor. The simple colon syntax is consistent with many other programming languages (see Appendix A), and it was heavily preferred by a cross section of Python developers who were surveyed.

Explicit Variance

We considered adding syntax for specifying whether a type parameter is intended to be invariant, covariant, or contravariant. The typing.TypeVar mechanism in Python requires this. A few other languages including Scala and C# also require developers to specify the variance. We rejected this idea because variance can generally be inferred, and most modern programming languages do infer variance based on usage. Variance is an advanced topic that many developers find confusing, so we want to eliminate the need to understand this concept for most Python developers.

Name Mangling

When considering implementation options, we considered a "name mangling" approach where each type parameter was given a unique "mangled" name by the compiler. This mangled name would be based on the qualified name of the generic class, function or type alias it was associated with. This approach was rejected because qualified names are not necessarily unique, which means the mangled name would need to be based on some other randomized value. Furthermore, this approach is not compatible with techniques used for evaluating quoted (forward referenced) type annotations.

Appendix A: Survey of Type Parameter Syntax

Support for generic types is found in many programming languages. In this section, we provide a survey of the options used by other popular programming languages. This is relevant because familiarity with other languages will make it easier for Python developers to understand this concept. We provide additional details here (for example, default type argument support) that may be useful when considering future extensions to the Python type system.

C++

C++ uses angle brackets in combination with keywords template and typename to declare type parameters. It uses angle brackets for specialization.

C++20 introduced the notion of generalized constraints, which can act like protocols in Python. A collection of constraints can be defined in a named entity called a concept.

Variance is not explicitly specified, but constraints can enforce variance.

A default type argument can be specified using the = operator.

Java

Java uses angle brackets to declare type parameters and for specialization. By default, type parameters are invariant. The extends keyword is used to specify an upper bound. The super keyword is used to specify a contravariant bound.

Java uses use-site variance. The compiler places limits on which methods and members can be accessed based on the use of a generic type. Variance is not specified explicitly.

Java provides no way to specify a default type argument.

C# --

C# uses angle brackets to declare type parameters and for specialization. The where keyword and a colon is used to specify the bound for a type parameter.

C# uses declaration-site variance using the keywords in and out for contravariance and covariance, respectively. By default, type parameters are invariant.

C# provides no way to specify a default type argument.

TypeScript

TypeScript uses angle brackets to declare type parameters and for specialization. The extends keyword is used to specify a bound. It can be combined with other type operators such as keyof.

TypeScript uses declaration-site variance. Variance is inferred from usage, not specified explicitly. TypeScript 4.7 introduced the ability to specify variance using in and out keywords. This was added to handle extremely complex types where inference of variance was expensive.

A default type argument can be specified using the = operator.

TypeScript supports the type keyword to declare a type alias, and this syntax supports generics.

Scala

In Scala, square brackets are used to declare type parameters. Square brackets are also used for specialization. The <: and >: operators are used to specify upper and lower bounds, respectively.

Scala uses use-site variance but also allows declaration-site variance specification. It uses a + or - prefix operator for covariance and contravariance, respectively.

Scala provides no way to specify a default type argument.

It does support higher-kinded types (type parameters that accept type type parameters).

Swift

Swift uses angle brackets to declare type parameters and for specialization. The upper bound of a type parameter is specified using a colon.

Swift doesn't support generic variance; all type parameters are invariant.

Swift provides no way to specify a default type argument.

Rust

Rust uses angle brackets to declare type parameters and for specialization. The upper bound of a type parameter is specified using a colon. Alternatively a where clause can specify various constraints.

Rust does not have traditional object oriented inheritance or variance. Subtyping in Rust is very restricted and occurs only due to variance with respect to lifetimes.

A default type argument can be specified using the = operator.

Kotlin

Kotlin uses angle brackets to declare type parameters and for specialization. By default, type parameters are invariant. The upper bound of a type is specified using a colon. Alternatively, a where clause can specify various constraints.

Kotlin supports declaration-site variance where variance of type parameters is explicitly declared using in and out keywords. It also supports use-site variance which limits which methods and members can be used.

Kotlin provides no way to specify a default type argument.

Julia

Julia uses curly braces to declare type parameters and for specialization. The <: operator can be used within a where clause to declare upper and lower bounds on a type.

Dart

Dart uses angle brackets to declare type parameters and for specialization. The upper bound of a type is specified using the extends keyword. By default, type parameters are covariant.

Dart supports declaration-site variance, where variance of type parameters is explicitly declared using in, out and inout keywords. It does not support use-site variance.

Dart provides no way to specify a default type argument.

Go --

Go uses square brackets to declare type parameters and for specialization. The upper bound of a type is specified after the name of the parameter, and must always be specified. The keyword any is used for an unbound type parameter.

Go doesn't support variance; all type parameters are invariant.

Go provides no way to specify a default type argument.

Go does not support generic type aliases.

Summary

| | Decl | Upper | Lower | Default | Variance | Variance | | | Syntax | Bound | Bound | Value | Site | |

| C++ | template | n/a | n/a | = | n/a | n/a | | | <> | | | | | |

| Java | <> | extends | | | use | super, | | | | | | | extends |

| C# | <> | where | | | decl | in, out |

| TypeScript | <> | extends | | = | decl | inferred, | | | | | | | in, out |

| Scala | [] | T <: X | T >: X | | use, decl | +, - |

| Swift | <> | T: X | | | n/a | n/a |

| Rust | <> | T: X, | | = | n/a | n/a | | | | where | | | | |

| Kotlin | <> | T: X, | | | use, decl | in, out | | | | where | | | | |

| Julia | {} | T <: X | X <: T | | n/a | n/a |

| Dart | <> | extends | | | decl | in, out, | | | | | | | | inout |

| Go | [] | T X | | | n/a | n/a |

| Python | [] | T: X | | | decl | inferred | | (proposed) | | | | | | |

Acknowledgements

Thanks to Sebastian Rittau for kick-starting the discussions that led to this proposal, to Jukka Lehtosalo for proposing the syntax for type alias statements and to Jelle Zijlstra, Daniel Moisset, and Guido van Rossum for their valuable feedback and suggested improvements to the specification and implementation.

Copyright

This document is placed in the public domain or under the CC0-1.0-Universal license, whichever is more permissive.